

STEP 1 — IMPORT LIBRARIES

```
import nltk
import re
import math
from collections import Counter
from nltk.util import ngrams
```

```
nltk.download('punkt')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt.zip.
True
```

STEP 2 — DATASET CREATE

```
dataset_text = """
Natural language processing is a subfield of artificial intelligence that focuses on the interaction between computers and h

A language model assigns probabilities to sequences of words. It helps the computer understand which sentences are more like

Unigram models consider each word independently. They do not use previous word context. Because of this unigram models are s

Trigram models consider three consecutive words. They capture more context than bigram models. However trigram models requir

One major issue in language modeling is unseen words. If a word does not appear in the training data its probability becomes

Add one smoothing also called Laplace smoothing adds one to all word counts. This ensures that no word has zero probability.

Perplexity is used to evaluate language models. Lower perplexity indicates a better model. Generally trigram models give low

Language models are used in many real world applications. They are used in mobile keyboards speech recognition systems chatb
"""

with open("dataset.txt", "w", encoding="utf-8") as f:
    f.write(dataset_text)
```

STEP 3 — LOAD DATASET

```
with open("dataset.txt", "r", encoding="utf-8") as f:
    text = f.read()

print(text[:200])
print("Total words:", len(text.split()))
```

```
Natural language processing is a subfield of artificial intelligence that focuses on the interaction between computers and h
Total words: 306
```

STEP 4 — PREPROCESS TEXT

```
def preprocess(text):
    text = text.lower()
    text = re.sub(r'^a-z\s', '', text)
    return text.split()

tokens = preprocess(text)
print(tokens[:20])
```

```
['natural', 'language', 'processing', 'is', 'a', 'subfield', 'of', 'artificial', 'intelligence', 'that', 'focuses', 'on', 't
```

Double-click (or enter) to edit

STEP 5 — BUILD N-GRAM MODELS Unigram

```
unigrams = Counter(tokens)
total_words = sum(unigrams.values())
```

Bigram

```
bigrams = Counter(ngrams(tokens, 2))
```

Trigram

```
trigrams = Counter(ngrams(tokens, 3))
```

STEP 6 — ADD-ONE (LAPLACE) SMOOTHING

```
vocab_size = len(unigrams)

def unigram_prob(w):
    return (unigrams[w] + 1) / (total_words + vocab_size)

def bigram_prob(w1, w2):
    return (bigrams[(w1, w2)] + 1) / (unigrams[w1] + vocab_size)

def trigram_prob(w1, w2, w3):
    return (trigrams[(w1, w2, w3)] + 1) / (bigrams[(w1, w2)] + vocab_size)
```

STEP 7 — SENTENCE PROBABILITY

```
def sentence_probability(sentence, model="bigram"):
    words = preprocess(sentence)
    prob = 1

    if model == "unigram":
        for w in words:
            prob *= unigram_prob(w)

    elif model == "bigram":
        for i in range(len(words)-1):
            prob *= bigram_prob(words[i], words[i+1])

    elif model == "trigram":
        for i in range(len(words)-2):
            prob *= trigram_prob(words[i], words[i+1], words[i+2])

    return prob
```

```
s = "language models are useful"
print(sentence_probability(s, "unigram"))
print(sentence_probability(s, "bigram"))
print(sentence_probability(s, "trigram"))
```

```
3.0486061727909776e-08
4.482214572576018e-06
0.00012093848262517134
```

STEP 8 — PERPLEXITY

```
def perplexity(sentence, model="bigram"):
    words = preprocess(sentence)
    N = len(words)
    log_prob = 0

    if model == "unigram":
        for w in words:
            log_prob += math.log(unigram_prob(w))

    elif model == "bigram":
        for i in range(len(words)-1):
            log_prob += math.log(bigram_prob(words[i], words[i+1]))

    elif model == "trigram":
        for i in range(len(words)-2):
```

```
log_prob += math.log(trigram_prob(words[i], words[i+1], words[i+2]))  
  
return math.exp(-log_prob / N)
```

Test:

```
print(perplexity(s, "unigram"))  
print(perplexity(s, "bigram"))  
print(perplexity(s, "trigram"))
```

```
75.67887523969229  
21.73335826355459  
9.535838147835364
```

```
print("Unigram size:", len(unigrams))  
print("Bigram size:", len(bigrams))  
print("Trigram size:", len(trigrams))
```

```
Unigram size: 154  
Bigram size: 268  
Trigram size: 295
```